TEMA: L1. Proiectarea studiului de caz:

Indicativ_echipa: **SIA_01**

STABILIRE surse de date externe

Descrieți fiecare sursă în parte într-un fișier sintetic trimis la adresa linus@uaic.ro (adăugați în mesaj toate fișierele de descriere și eventual creare/configurare necesare):

- **DS_1**: Payments System – sursă de date tranzacțională importată în PostgreSQL
  - Tip sursă de date
    - Tip model de date: relațional
    - Tip format de access: SQL
  - Descriere structuri de date

1. cards

- card_id – identificator card
- client_id – identificator client
- card_brand – brand card
- card_type – tip card
- expires – data expirării
- acct_open_date – data deschiderii contului/cardului
- num_cards_issued – număr de carduri emise

2. card_limits

- card_id – identificator card
- credit_limit – limită de credit

3. card_security

- card_id – identificator card
- has_chip – indicator existență cip
- year_pin_last_changed – anul ultimei modificări a PIN-ului
- card_on_dark_web – indicator expunere pe dark web

4. merchants

- merchant_id – identificator comerciant
- merchant_city – oraș comerciant
- merchant_state – stat/regiune comerciant
- zip – cod poștal

5. transactions

- txn_id – identificator tranzacție

            - date – data tranzacției
            - client_id – identificator client
            - card_id – identificator card
            - amount – valoare tranzacție
            - use_chip – mod utilizare card
            - merchant_id – identificator comerciant
            - mcc – cod categorie comerciant
            - errors – eventuale erori asociate tranzacției
        - legături între structurile din cadrul aceleași surse de date.
            - cards.card_id = card_limits.card_id
            - cards.card_id = card_security.card_id
            - transactions.card_id = cards.card_id
            - transactions.merchant_id = merchants.merchant_id
    - Implementare sursa de date externa

      TextFile.csv
        - cards.csv
        - card_limits.csv
        - card_security.csv
        - transactions_clean.csv
        - merchants.csv

      Script Comenzi DDL DML:
        - creare rol și schemă payments

```
DO $$
BEGIN
IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'payments') THEN
CREATE ROLE payments WITH
LOGIN
NOSUPERUSER
NOCREATEDB
NOCREATEROLE
INHERIT
NOREPLICATION
CONNECTION LIMIT -1
PASSWORD 'payments';
END IF;
END $$;
```

```
CREATE SCHEMA IF NOT EXISTS payments AUTHORIZATION payments;

SELECT n.nspname AS schema, r.rolname AS owner
FROM pg_namespace n
JOIN pg_roles r ON r.oid = n.nspowner
WHERE n.nspname = 'payments';
```

- creare tabele staging

```
DROP TABLE IF EXISTS payments.stg_cards;
DROP TABLE IF EXISTS payments.stg_card_limits;
DROP TABLE IF EXISTS payments.stg_card_security;
DROP TABLE IF EXISTS payments.stg_merchants;
DROP TABLE IF EXISTS payments.stg_transactions;

CREATE TABLE payments.stg_cards (
    card_id TEXT,
    client_id TEXT,
    card_brand TEXT,
    card_type TEXT,
    expires TEXT,
    acct_open_date TEXT,
    num_cards_issued TEXT
);

CREATE TABLE payments.stg_card_limits (
    card_id TEXT,
    credit_limit TEXT
);

CREATE TABLE payments.stg_card_security (
    card_id TEXT,
    has_chip TEXT,
    year_pin_last_changed TEXT,
    card_on_dark_web TEXT
);
```

```
CREATE TABLE payments.stg_merchants (
   merchant_id TEXT,
   merchant_city TEXT,
   merchant_state TEXT,
   zip TEXT
);

CREATE TABLE payments.stg_transactions (
   txn_id TEXT,
   date TEXT,
   client_id TEXT,
   card_id TEXT,
   amount TEXT,
   use_chip TEXT,
   merchant_id TEXT,
   mcc TEXT,
   errors TEXT
);
```

- import CSV în tabele staging

```
\copy payments.stg_cards FROM 'path\cards.csv' WITH (FORMAT csv, HEADER true)
\copy payments.stg_card_limits FROM 'path\card_limits.csv' WITH (FORMAT csv, HEADER true)
\copy payments.stg_card_security FROM 'path\card_security.csv' WITH (FORMAT csv, HEADER true)
\copy payments.stg_merchants FROM 'path\merchants.csv' WITH (FORMAT csv, HEADER true)
\copy payments.stg_transactions FROM 'path\transactions_clean.csv' WITH (FORMAT csv, HEADER true)
```

- creare tabele finale relaționale

```
DROP TABLE IF EXISTS payments.transactions_clean;
DROP TABLE IF EXISTS payments.card_security;
DROP TABLE IF EXISTS payments.card_limits;
DROP TABLE IF EXISTS payments.merchants;
```

```sql
DROP TABLE IF EXISTS payments.cards;

CREATE TABLE payments.cards (
   card_id INTEGER PRIMARY KEY,
   client_id INTEGER NOT NULL,
   card_brand VARCHAR(20),
   card_type VARCHAR(30),
   expires DATE,
   acct_open_date DATE,
   num_cards_issued INTEGER
);

CREATE TABLE payments.merchants (
   merchant_id INTEGER PRIMARY KEY,
   merchant_city VARCHAR(100),
   merchant_state VARCHAR(50),
   zip VARCHAR(50)
);

CREATE TABLE payments.card_limits (
   card_id INTEGER PRIMARY KEY,
   credit_limit NUMERIC(12,2),
   CONSTRAINT fk_limits_card
      FOREIGN KEY (card_id)
      REFERENCES payments.cards(card_id)
);

CREATE TABLE payments.card_security (
   card_id INTEGER PRIMARY KEY,
   has_chip BOOLEAN,
   year_pin_last_changed INTEGER,
   card_on_dark_web BOOLEAN,
   CONSTRAINT fk_security_card
      FOREIGN KEY (card_id)
      REFERENCES payments.cards(card_id)
);
```

```
CREATE TABLE payments.transactions_clean (
   txn_id BIGINT PRIMARY KEY,
   date TIMESTAMP,
   client_id INTEGER,
   card_id INTEGER,
   amount NUMERIC(12,2),
   use_chip VARCHAR(30),
   merchant_id INTEGER,
   mcc INTEGER,
   errors VARCHAR(255),
   CONSTRAINT fk_tx_card
      FOREIGN KEY (card_id)
      REFERENCES payments.cards(card_id),
   CONSTRAINT fk_tx_merchant
      FOREIGN KEY (merchant_id)
      REFERENCES payments.merchants(merchant_id)
);
```

- conversie și curățare date (cast, eliminare $, conversii date calendaristice)

```
INSERT INTO payments.cards
SELECT
   CAST(card_id AS INTEGER),
   CAST(client_id AS INTEGER),
   card_brand,
   card_type,
   TO_DATE(expires, 'MM/YYYY'),
   TO_DATE(acct_open_date, 'MM/YYYY'),
   CAST(num_cards_issued AS INTEGER)
FROM payments.stg_cards;

INSERT INTO payments.merchants
SELECT
   CAST(merchant_id AS INTEGER),
   merchant_city,
```

```
    merchant_state,
    zip
FROM payments.stg_merchants;

INSERT INTO payments.card_limits
SELECT
    CAST(card_id AS INTEGER),
    CAST(
        REPLACE(REPLACE(credit_limit, '$',''), ',', '')
        AS NUMERIC(12,2)
    )
FROM payments.stg_card_limits;

INSERT INTO payments.card_security
SELECT
    CAST(card_id AS INTEGER),
    CASE WHEN UPPER(has_chip) = 'YES' THEN TRUE ELSE FALSE END,
    CAST(year_pin_last_changed AS INTEGER),
    CASE WHEN UPPER(card_on_dark_web) = 'YES' THEN TRUE ELSE FALSE END
FROM payments.stg_card_security;

INSERT INTO payments.transactions_clean
SELECT
    CAST(txn_id AS BIGINT),
    CAST(date AS TIMESTAMP),
    CAST(client_id AS INTEGER),
    CAST(card_id AS INTEGER),
    CAST(amount AS NUMERIC(12,2)),
    use_chip,
    CAST(merchant_id AS INTEGER),
    CAST(mcc AS INTEGER),
    errors
FROM payments.stg_transactions;
```

- **DS_2: CRM Customers – sursă de date tabelară din fișier Excel**
    - Tip sursă de date

        - Tip model de date: tabelar / relațional simplificat
        - Tip format de access: XLSX
    - Descriere structuri de date

1. customers

- client_id – identificator client
- current_age – vârsta curentă
- retirement_age – vârsta estimată de pensionare
- birth_year – anul nașterii
- birth_month – luna nașterii
- gender – gen
- address – adresă
- latitude – latitudine
- longitude – longitudine

2. customer_finance

- client_id – identificator client
- per_capita_income – venit per capita
- yearly_income – venit anual
- total_debt – datorie totală
- credit_score – scor de credit
- num_credit_cards – număr carduri de credit

        - legături între structurile din cadrul aceleași surse de date.
            - customers.client_id = customer_finance.client_id
    - Implementare sursa de date externa

Document.XLSx

- crm.xlsx
- sheet customers
- sheet customer_finance

Script Comenzi DDL DML:

- creare tabel CRM.CUSTOMERS

```
SHOW CON_NAME;
ALTER SESSION SET CONTAINER = XEPDB1;
SHOW CON_NAME;

BEGIN
  EXECUTE IMMEDIATE 'DROP USER FDBO CASCADE';
EXCEPTION WHEN OTHERS THEN NULL;
```

```
END;
/

CREATE USER FDBO IDENTIFIED BY fdbo
DEFAULT TABLESPACE users
TEMPORARY TABLESPACE temp
QUOTA UNLIMITED ON USERS;

GRANT CONNECT, RESOURCE TO FDBO;
GRANT CREATE VIEW TO FDBO;
GRANT CREATE DATABASE LINK TO FDBO;
GRANT CREATE ANY DIRECTORY TO FDBO;
GRANT EXECUTE ON UTL_HTTP TO FDBO;
GRANT EXECUTE ON DBMS_LOB TO FDBO;
GRANT EXECUTE ON SYS.DBMS_CRYPTO TO FDBO;
GRANT CREATE DIMENSION, CREATE JOB, CREATE MATERIALIZED VIEW, CREATE
SYNONYM TO FDBO;

BEGIN
  DBMS_NETWORK_ACL_ADMIN.APPEND_HOST_ACE (
      host       => '*',
      lower_port => NULL,
      upper_port => NULL,
      ace        => XS$ACE_TYPE(
                    privilege_list => XS$NAME_LIST('http'),
                    principal_name => 'FDBO',
                    principal_type => XS_ACL.PTYPE_DB
                  )
  );
END;
/
COMMIT;
```

- Curățare și conversie date financiare

```
CREATE TABLE CRM.CUSTOMER_FINANCE_CLEAN AS
SELECT
```

```
  client_id,
  TO_NUMBER(REPLACE(REPLACE(per_capita_income, '$',''), ',','')) AS per_capita_income,
  TO_NUMBER(REPLACE(REPLACE(yearly_income, '$',''), ',',''))     AS yearly_income,
  TO_NUMBER(REPLACE(REPLACE(total_debt, '$',''), ',',''))        AS total_debt,
  credit_score,
  num_credit_cards
FROM CRM.CUSTOMER_FINANCE;
```

- Definire constrângeri

```
ALTER TABLE CRM.CUSTOMERS
  ADD CONSTRAINT PK_CUSTOMERS PRIMARY KEY (client_id);

ALTER TABLE CRM.CUSTOMER_FINANCE
  ADD CONSTRAINT PK_CUSTOMERS_FIN PRIMARY KEY (client_id);

ALTER TABLE CRM.CUSTOMER_FINANCE
  ADD CONSTRAINT FK_FIN_CUSTOMER
  FOREIGN KEY (client_id) REFERENCES CRM.CUSTOMERS(client_id);

GRANT SELECT ON CRM.CUSTOMERS TO FDBO;
GRANT SELECT ON CRM.CUSTOMER_FINANCE TO FDBO;
```

- DS_3
  - Tip sursă de date
    - Tip model de date: documentar / ierarhic
    - Tip format de access: JSON
  - Descriere structuri de date

1. customer_risk_docs

Exemple de atribute:

- client_id – identificator client
- credit_score – scor de credit
- yearly_income – venit anual
- total_debt – datorie totală
- num_credit_cards – număr carduri
- tx_count – număr tranzacții
- total_amount – sumă totală tranzacționată

- top_mcc – cod MCC dominant

2. transaction_docs

Exemple de atribute:

- txn_id – identificator tranzacție
- ts – timestamp tranzacție
- client_id – identificator client
- card_id – identificator card
- amount – valoare tranzacție
- channel – canal tranzacție
- merchant – subdocument comerciant:
  - id
  - city
  - state
  - zip
- mcc – cod categorie comerciant

- legături între structurile din cadrul aceleași surse de date.
  - customer_risk_docs.client_id se corelează logic cu transaction_docs.client_id
  - transaction_docs.card_id permite corelare cu entitățile cardurilor din sursa tranzacțională
  - transaction_docs.merchant.id permite corelare cu comercianții

- Implementare sursa de date externa

Document.JSON

- customer_risk_docs.jsonl
- transaction_docs.jsonlScript

Comenzi DDL DML:

- creare bază de date MongoDB mds

```
use mds
show dbs
```

- import colecții cu mongoimport

```
mongoimport --db mds --collection customer_risk_docs --file "PATH"

mongoimport --db mds --collection transaction_docs --file " PATH "
```

- validare colecții și număr de documente

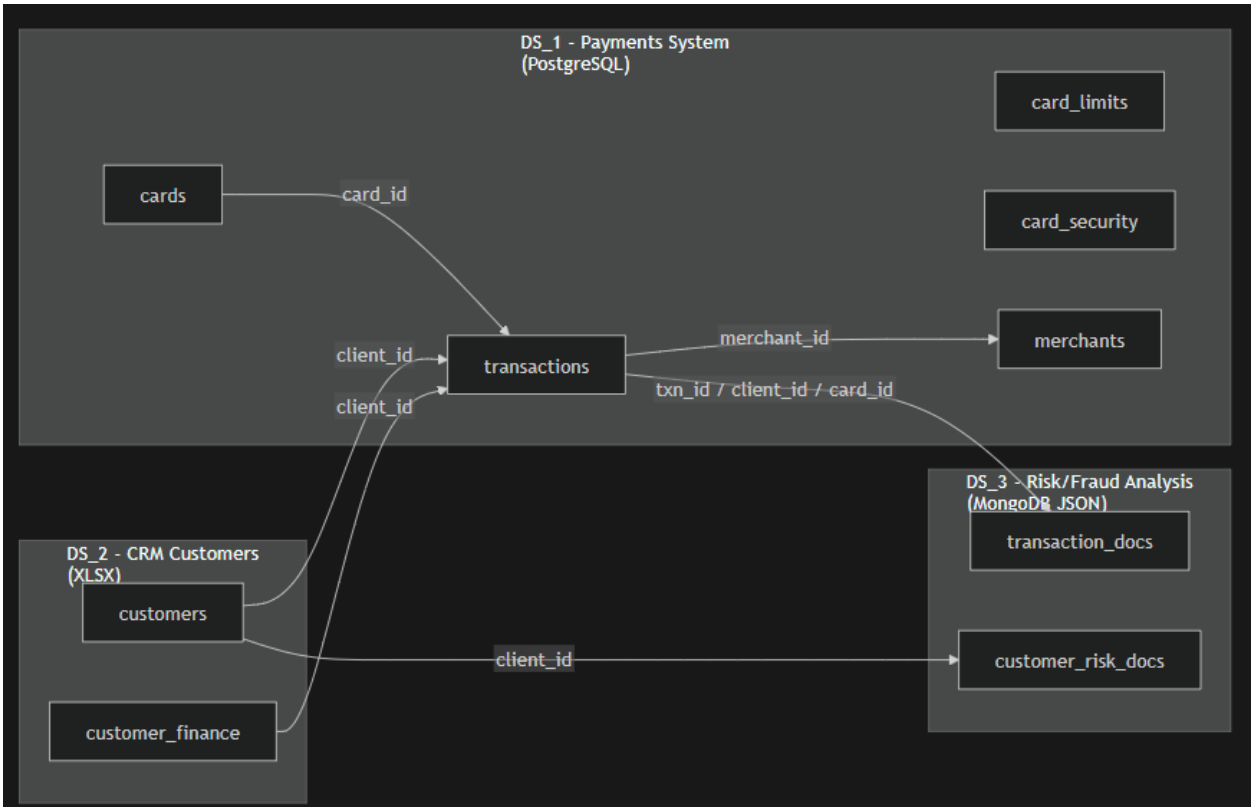
mongosh
use mds
show collections
db.customer_risk_docs.countDocuments()
db.transaction_docs.countDocuments()

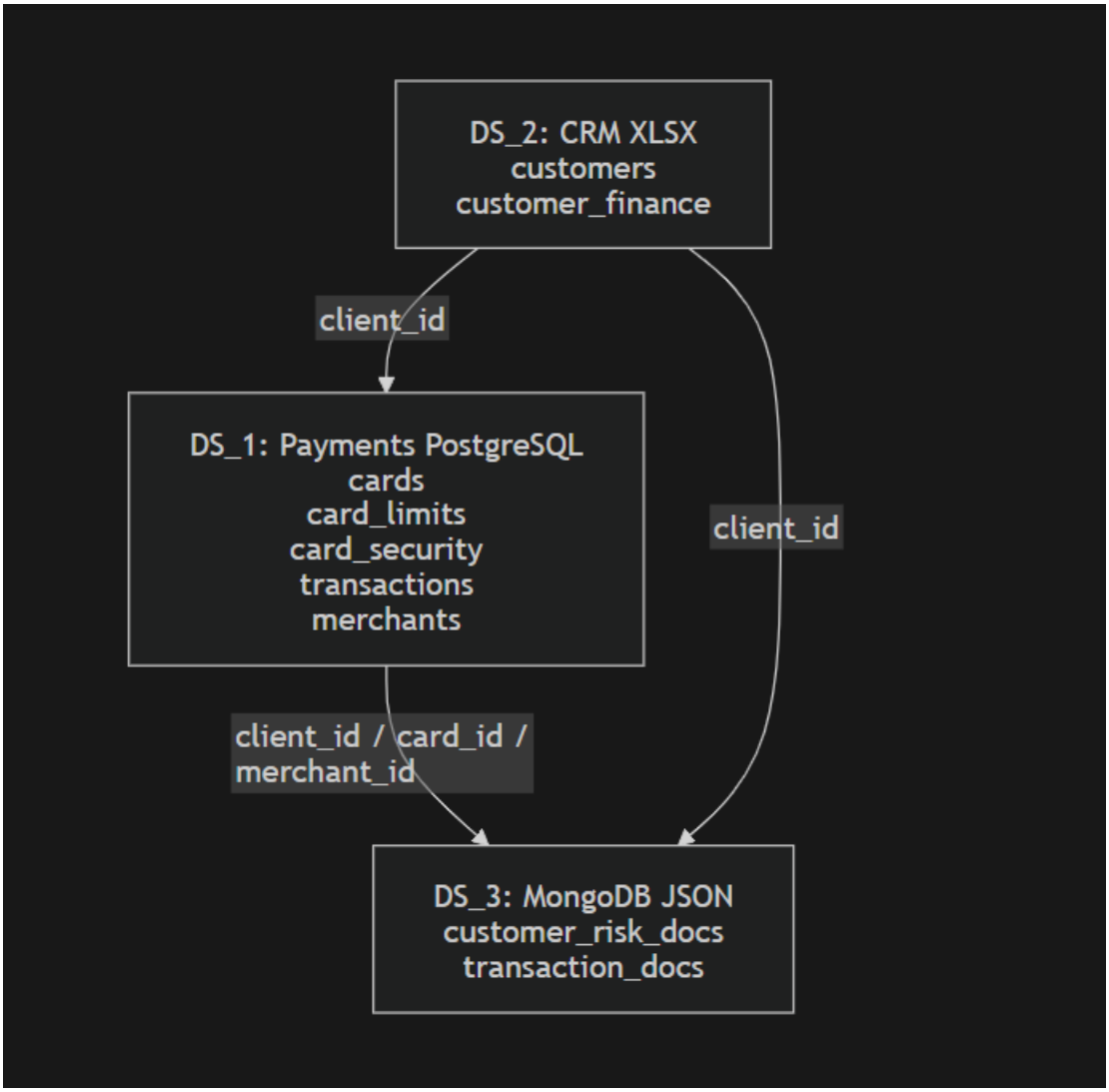
- expunere prin RESTHeart pentru acces extern

cd C:\Users\savas\restheart\restheart
java -jar restheart.jar -o conf-override.conf

http://localhost:8081/mds
http://localhost:8081/mds/customer_risk_docs
http://localhost:8081/mds/transaction_docs

Diagrama




DS_2: CRM XLSX
customers
customer_finance
client_id
DS_1: Payments PostgreSQL
cards
card_limits
card_security
transactions
merchants
client_id
client_id / card_id /
merchant_id
DS_3: MongoDB JSON
customer_risk_docs
transaction_docs